

Franka Tilted Regrasp — Deployment Reference

Action and observation contract for **FrankaEnvParallelTiltedNoForce**

Source: `examples/rigid/env_franka_parallel_tilted_no_force.py` · captured at tilt 45°, seed 1, dt 0.001 s, control period 0.02 s (50 Hz)

1 · Coordinate frame

Every quantity named z lives in the **tilted task frame**, not in world coordinates. The frame is a fixed rotation captured once per episode from the home pose; at tilt 0° it equals world XYZ, and at 45° its Z axis is the grasp line inclined 45° from world vertical toward approximately world +Y. Positions are reported as $\text{HOME_TASK_Z} + (p - \text{task_origin}) \cdot \hat{z}$ and velocities as $\dot{v} \cdot \hat{z}$.

On hardware you must reproduce this projection. Feeding raw world-frame z into a policy trained at 45° will misread both position and velocity by a factor of $\cos(45^\circ) \approx 0.707$ along the intended axis, plus cross-coupling from the transverse components. Gravity is **not** rotated: it stays world-down in simulation, so the tilted axis is not a gravity-aligned axis.

$\text{HOME_TASK_Z} = 0.854586497 \text{ m}$ — the nominal FK hand height used as the task-frame origin offset.

2 · Action space

Shape (N, 3), every channel expected in $[-1, 1]$ and hard-clamped to that range on entry. Out-of-range values are silently saturated, not rejected.

#	Channel	Maps to	Range after scaling
0	target_z_vel	commanded velocity along the tilted grasp axis	$\pm 0.45 \text{ m/s}$ (Z_VEL_MAX)
1	finger_l	left finger position command	0.000251 → 0.0130 m
2	finger_r	right finger position command	0.000251 → 0.0130 m

2.1 Velocity channel

$v_{\text{cmd}} = \text{clamp}(a[0], -1, 1) \times 0.45$, then rate-limited to $|\Delta v| \leq Z_ACC_MAX \times \text{target_dt} = 15.0 \times 0.02 = 0.30 \text{ m/s}$ per control step. The limiter is stateful: it clamps against the *previous* commanded velocity, so a step from -1 to $+1$ takes three control steps (60 ms) to take full effect. A deployment controller must apply the same limiter, or the policy's implicit acceleration model will not match the plant.

2.2 Gripper channels — trigger, not a position

The two finger channels are averaged and compared against the midpoint of the finger range. A **rising edge** through that midpoint fires a fixed-shape pulse; outside a pulse the fingers are clamped fully closed no matter what the policy commands. The policy therefore cannot hold the gripper open — it can only request a release.

Phase	Duration	Finger command
Delay (PULSE_DELAY_STEPS = 2)	40 ms	policy value (already 'open')
Forced open (PULSE_LENGTH - 1 = 6)	120 ms	gripper_pos_max, policy overridden
Forced close (1 step)	20 ms	gripper_pos_min, policy overridden
Lockout (PULSE_LOCKOUT_DURATION)	1.2 s from trigger	clamped closed; no new pulse

No pulse may fire in the first $\text{PULSE_START_MIN_STEPS} = 50$ control steps (1.0 s) of an episode. Total loose time per pulse is 160 ms, of which the last 120 ms are uninterruptible.

3 · Observation space

Shape (N, 8), dtype float32. This variant deliberately **excludes finger contact forces** — the reward and termination logic still read them internally, but the policy never sees them, so no force sensing is required at deployment.

#	Name	Meaning	Unit	Source	OBS_SCALE
0	ee_pos_z	EE position projected on the tilted axis, as HOME_TASK_Z + displacement	m	measured	1
1	ee_vel_z	EE velocity projected on the tilted axis	m/s	measured	0.6
2	target_z_vel	commanded axial velocity currently in effect (post rate-limit)	m/s	internal	0.6
3	target_z_acc	(target_z_vel – previous target_z_vel) / target_dt	m/s ²	internal	15
4	cuboid_rel_z	cuboid centre minus fingertip midpoint, axial component	m	measured	0.05
5	cuboid_rel_x	same vector, transverse X component	m	measured	0.05
6	cuboid_rel_y	same vector, transverse Y component	m	measured	0.05
7	desired_rel_z	commanded grasp offset for this episode (the goal)	m	command	0.05

3.1 Normalization

With `--normalization` (used for this capture) the whole vector is divided elementwise by `OBS_SCALE` as the final step of observation assembly. The deployment stack must apply the identical division, in the same order, before the policy call.

```
OBS_SCALE = [1.0, 0.6, 0.6, 15.0, 0.05, 0.05, 0.05, 0.05]
obs_policy = obs_raw / OBS_SCALE
```

Note channel 0 has scale 1.0, so the normalized value still carries the ~0.85 m absolute offset — it is not centred. Do not substitute a zero-centred height.

3.2 Observation noise

With `randomize=True`, ±1 mm uniform noise is added to the five measured channels (0, 1, 4, 5, 6 — position-derived) before normalization. The three internal/command channels (2, 3, 7) are always noise-free. This capture used `randomize=False`, so the values below are clean.

4 · Reset state — 10 consecutive resets

Captured from the live environment (Genesis 0.4.6, CUDA) with `seed=1`, `tilt_deg=45`, `num_envs=1`, `normalize=True`, `randomize=False`. Values are **raw, pre-normalization** in SI units. Reproduce with `reports/deploy_spec/_collect_resets.py`.

reset	ee_pos_z	ee_vel_z	target_z_vel	target_z_acc	cuboid_rel_z	cuboid_rel_x	cuboid_rel_y	desired_rel_z
0	0.852748	-0.002565	0	0	-5.9e-05	-1e-06	6e-06	0.01469
1	0.852748	-0.002565	0	0	-5.7e-05	-1e-06	5e-06	0.015595
2	0.852748	-0.002565	0	0	-5.1e-05	0	3e-06	0.014684
3	0.852748	-0.002565	0	0	-5.1e-05	0	3e-06	0.014549
4	0.852748	-0.002565	0	0	-5.1e-05	0	3e-06	0.014511
5	0.852748	-0.002565	0	0	-6e-05	-1e-06	7e-06	0.021978
6	0.852748	-0.002565	0	0	-6.2e-05	-1e-06	7e-06	0.016667
7	0.852748	-0.002565	0	0	-6.2e-05	-1e-06	7e-06	0.014731
8	0.852748	-0.002565	0	0	-6e-05	-1e-06	7e-06	0.014284
9	0.852748	-0.002565	0	0	-5.4e-05	-1e-06	4e-06	0.017637

4.1 What varies and what does not

The physical start state is deterministic. Across all ten resets, channels 0–6 are identical to within 1e-5: the arm returns to the same home configuration and the cuboid is placed analytically in the fingers, not dropped or settled. The

only channel that changes is `desired_rel_z`, which is freshly sampled each episode from $U[0.0125, 0.0225]$ m.

Channel	Value at reset	Varies across resets?
<code>ee_pos_z</code>	0.852748 m	no (identical every reset)
<code>ee_vel_z</code>	-0.002565 m/s	no — residual drift from the 100-step warmup, not zero
<code>target_z_vel</code>	0	no — explicitly zeroed
<code>target_z_acc</code>	0	no — explicitly zeroed
<code>cuboid_rel_z</code>	$\approx -5.7e-05$ m	negligible ($\pm 6e-06$)
<code>cuboid_rel_x</code>	$\approx -1e-06$ m	negligible
<code>cuboid_rel_y</code>	$\approx +5e-06$ m	negligible
<code>desired_rel_z</code>	0.01428 – 0.02198 m	yes — sampled $U[0.0125, 0.0225]$ per episode

Consequence for deployment. To reproduce a given episode you only need to pin one number: `desired_rel_z`. Everything else follows from homing the arm and seating the object. Note that `ee_pos_z` at reset (0.852748) sits slightly below `HOME_TASK_Z` (0.854587) and that `ee_vel_z` is not exactly zero — the environment runs a 100-step warmup before handing over the first observation, and the arm has not fully settled. If your hardware starts from a dead stop, the policy sees a marginally different first observation than it did in training.

5 • Control loop contract

Property	Value
Policy rate	50 Hz (<code>target_dt</code> = 0.02 s)
Physics rate	1000 Hz (<code>dt</code> = 0.001 s), 20 substeps per policy step
Episode length	450 policy steps = 9.0 s
Action latency	action applied at the start of the substep block, held for all 20 substeps
Observation timing	read after the substep block completes

The environment re-applies the action inside the substep loop, so the effective command is held zero-order over the full 20 ms. A deployment controller running the policy at a different rate must rescale `Z_ACC_MAX` $\times$ `target_dt` accordingly, or the rate limiter will behave differently from training.

5.1 Minimal inference loop

```
obs_raw = [
    HOME_TASK_Z + dot(ee_pos - task_origin, z_hat), # 0
    dot(ee_vel, z_hat), # 1
    v_cmd_prev, # 2
    (v_cmd_prev - v_cmd_prev2) / 0.02, # 3
    dot(cuboid_pos - finger_mid, z_hat), # 4
    dot(cuboid_pos - finger_mid, x_hat), # 5
    dot(cuboid_pos - finger_mid, y_hat), # 6
    desired_rel_z, # 7
]
a = policy(obs_raw / OBS_SCALE)
v_cmd = clamp(clamp(a[0], -1, 1) * 0.45,
              v_cmd_prev - 0.30, v_cmd_prev + 0.30)
if rising_edge(mean(a[1:]) > midpoint) and not locked_out:
    fire_pulse() # 40 ms delay, 120 ms open, 20 ms close
```

A GRU policy additionally carries hidden state across steps; it must be zeroed on episode reset and never zeroed mid-episode.

6 • Termination bounds

These are the simulator's episode-ending conditions. They are **not** safety limits, but they do mark the region the policy was trained inside — outside it, behaviour is undefined.

Condition	Threshold	Outcome
<code> cuboid_rel_x </code> or <code> cuboid_rel_y </code>	> 0.04 m	fail
<code> cuboid_rel_z </code>	> 0.15 m	fail

Condition	Threshold	Outcome
fingertip separation	< 0.01 m	fail
ee_pos_z	< 0.60 m or > 0.96 m	fail
regrasp count	reaches 7	fail
episode length	450 steps	timeout
$ \text{cuboid_rel_z} - \text{desired_rel_z} \leq 0.005$ m, settled and firmly grasped for 5 steps	see SUCCESS_* constants	success

Success additionally requires $|\text{ee_vel_z}| < 0.05$ m/s, $0.70 \leq \text{ee_pos_z} \leq 0.86$ m and three consecutive steps of firm grasp.

7 · Deployment checklist

✓	Item
1	Build the tilted task frame from the home pose and project <i>all</i> z quantities through it.
2	Divide the observation by OBS_SCALE — the same vector, the same order.
3	Keep v_cmd_prev to produce channels 2 and 3; they are the commanded values, not measured ones.
4	Apply the 0.30 m/s per-step velocity rate limit before sending the command.
5	Treat the gripper output as an edge-triggered pulse request, not a position; enforce the 1.2 s lockout and the closed clamp between pulses.
6	Pin desired_rel_z to reproduce a specific episode; the rest of the start state is deterministic.
7	Zero the GRU hidden state on reset only.
8	No force sensing is needed — this variant removed contact forces from the observation.

Generated from the live environment; reward-shaping constants are omitted because they do not affect inference. Constants quoted here were read from the source at capture time and will drift if the environment file changes — regenerate with `reports/deploy_spec/make_report.py`.